Introduction

Python provides several built-in data types for storing and organizing data. This document covers five important collection and sequence types: String, List, Tuple, Set, and Dictionary. For each type, the definition, major properties, important built-in methods, and practical examples are provided.

Quick Comparison

Data Type	Ordered	Mutable	Duplicates	Syntax	Main Use
String	Yes	No	Yes	'Hello'	Text
List	Yes	Yes	Yes	[1, 2, 3]	Changeable collection
Tuple	Yes	No	Yes	(1, 2, 3)	Fixed collection
Set	No*	Yes	No	{1, 2, 3}	Unique items / set operations
Dictionary	Yes**	Yes	Keys: No	{'a': 1}	Key-value data

* Sets are unordered collections; their iteration order should not be relied upon.

** Dictionaries preserve insertion order in modern Python (Python 3.7+).

2. String

Definition

A string is an immutable sequence of Unicode characters used to store text. Strings can be created using single quotes, double quotes, or triple quotes.

Properties / Characteristics

- Ordered: characters have a definite position (index).
- Immutable: individual characters cannot be changed after creation.
- Supports indexing and slicing.
- Allows duplicate characters.
- Supports many text-processing and searching methods.
- Strings can contain letters, numbers, symbols, spaces, and Unicode characters.

Important Methods

Method	Purpose	Example
capitalize()	Converts the first character to uppercase.	"hello".capitalize() → "Hello"
casefold()	Converts text to lowercase for case-insensitive comparisons.	"PYTHON".casefold() → "python"
center(width)	Centers the string within the specified width.	"Hi".center(6) → " Hi "
count(sub)	Counts non-overlapping occurrences of a substring.	"banana".count("a") → 3
encode()	Encodes the string into bytes.	"Hi".encode()
endswith(suffix)	Checks whether the string ends with a value.	"file.py".endswith(".py") → True
find(sub)	Returns the first index of a substring;	"python".find("th") → 2

	returns -1 if absent.	
format()	Formats values into a string.	"Hello {}".format("Sam") → "Hello Sam"
index(sub)	Returns the first index; raises ValueError if absent.	"python".index("th") → 2
isalnum()	Checks whether all characters are alphanumeric.	"Python3".isalnum() → True
isalpha()	Checks whether all characters are alphabetic.	"Python".isalpha() → True
isdigit()	Checks whether all characters are digits.	"123".isdigit() → True
islower()	Checks whether all cased characters are lowercase.	"hello".islower() → True
isspace()	Checks whether all characters are whitespace.	" ".isspace() → True
istitle()	Checks whether the string is title-cased.	"Hello World".istitle() → True
isupper()	Checks whether all cased characters are uppercase.	"HELLO".isupper() → True
join(iterable)	Joins iterable items using the string as a separator.	"-".join(["A","B"]) → "A-B"
lower()	Converts the string to lowercase.	"PYTHON".lower() → "python"
lstrip()	Removes leading whitespace or specified characters.	" hi".lstrip() → "hi"
partition(sep)	Splits into three parts around the first separator.	"a-b".partition("-")
replace(old, new)	Replaces occurrences of a substring.	"cat".replace("c","b") → "bat"
rfind(sub)	Returns the highest index of a substring.	"banana".rfind("a") → 5
rindex(sub)	Returns the highest index; raises ValueError if absent.	"banana".rindex("a") → 5
rsplit()	Splits from the right.	"a-b-c".rsplit("-", 1) → ["a-b","c"]
rstrip()	Removes trailing whitespace or specified characters.	"hi ".rstrip() → "hi"
split()	Splits a string into a list.	"a,b".split(",") → ["a","b"]
splitlines()	Splits a string at line boundaries.	"a\nb".splitlines() → ["a","b"]
startswith(prefix)	Checks whether the string starts with a value.	"Python".startswith("Py") → True
strip()	Removes leading and trailing whitespace or characters.	" hi ".strip() → "hi"
swapcase()	Swaps uppercase and lowercase characters.	"PyThOn".swapcase() → "pYtHoN"
title()	Converts words to title case.	"hello world".title() → "Hello World"
upper()	Converts the string to uppercase.	"python".upper() → "PYTHON"
zfill(width)	Pads a numeric string on the left with zeros.	"42".zfill(5) → "00042"

Examples

Example 1: Basic string operations

```
name = "Python"
print(name[0])       # P
```

```
print(name[0:3])     # Pyt
print(name.upper())  # PYTHON
```

Example 2: Cleaning and splitting text

```
text = "  Python,Java,C++  "
clean = text.strip()
items = clean.split(",")
print(items)          # ["Python", "Java", "C++"]
```

3. List

Definition

A list is an ordered and mutable collection that can store multiple values. Lists can contain values of different data types.

Properties / Characteristics

- Ordered and indexed.
- Mutable: items can be added, removed, or changed.
- Allows duplicate values.
- Supports positive and negative indexing.
- Supports slicing.
- Can contain mixed data types and nested lists.

Important Methods

Method	Purpose	Example
append(x)	Adds one item to the end.	a.append(4)
clear()	Removes all items.	a.clear()
copy()	Returns a shallow copy.	b = a.copy()
count(x)	Counts occurrences of a value.	a.count(2)
extend(iterable)	Adds all items from an iterable.	a.extend([4,5])
index(x)	Returns the first index of a value.	a.index(2)
insert(i, x)	Inserts an item at a given position.	a.insert(1, 10)
pop([i])	Removes and returns an item; last by default.	x = a.pop()
remove(x)	Removes the first matching item.	a.remove(2)
reverse()	Reverses the list in place.	a.reverse()
sort()	Sorts the list in place.	a.sort()

Examples

Example 1: Adding and modifying items

```
numbers = [10, 20, 30]
numbers.append(40)
numbers.insert(1, 15)
numbers[0] = 5
print(numbers)  # [5, 15, 20, 30, 40]
```

Example 2: Sorting and removing items

```
marks = [75, 40, 90, 60]
marks.sort()
marks.remove(60)
print(marks)   # [40, 75, 90]
```

4. Tuple

Definition

A tuple is an ordered collection of values that is immutable. Tuples are commonly used for fixed groups of related data.

Properties / Characteristics

- Ordered and indexed.
- Immutable: elements cannot be added, removed, or changed.
- Allows duplicate values.
- Supports indexing and slicing.
- Can contain mixed data types.
- Usually written with parentheses, such as (1, 2, 3).
- Can be used as a dictionary key when all contained elements are hashable.

Important Methods

Method	Purpose	Example
count(x)	Returns the number of occurrences of a value.	t.count(2)
index(x)	Returns the first index of a value.	t.index(2)

Examples

Example 1: Indexing and unpacking

```
student = ("Abhinav", 24, "MTech")
print(student[0])
name, age, course = student
print(name, age, course)
```

Example 2: Using tuple methods

```
t = (10, 20, 10, 30, 10)
print(t.count(10))  # 3
print(t.index(20))  # 1
```

5. Set

Definition

A set is a mutable, unordered collection of unique elements. It is useful when duplicate values must be removed or when mathematical set operations are required.

Properties / Characteristics

- Unordered: no reliable positional indexing.
- Mutable: elements can be added and removed.
- Does not allow duplicate elements.
- Elements must be hashable.
- Supports union, intersection, difference, and symmetric difference.
- An empty set is created with set(), not {} ({} creates an empty dictionary).
- A frozenset is the immutable counterpart of a set.

Important Methods

Method	Purpose	Example
add(x)	Adds an element.	s.add(4)
clear()	Removes all elements.	s.clear()
copy()	Returns a shallow copy.	t = s.copy()
difference(other)	Returns elements present in the first set but not the other.	a.difference(b)
difference_update(other)	Removes elements found in another set.	a.difference_update(b)
discard(x)	Removes an element if present; no error if absent.	s.discard(2)
intersection(other)	Returns common elements.	a.intersection(b)
intersection_update(other)	Keeps only common elements.	a.intersection_update(b)
isdisjoint(other)	Checks whether two sets have no common elements.	a.isdisjoint(b)
issubset(other)	Checks whether all elements are contained in another set.	a.issubset(b)
issuperset(other)	Checks whether the set contains another set.	a.issuperset(b)
pop()	Removes and returns an arbitrary element.	x = s.pop()
remove(x)	Removes an element; raises KeyError if absent.	s.remove(2)
symmetric_difference(other)	Returns elements in either set but not both.	a.symmetric_difference(b)
symmetric_difference_update(other)	Updates the set to its symmetric difference.	a.symmetric_difference_update(b)
union(other)	Returns all unique elements from both sets.	a.union(b)
update(iterable)	Adds elements from an iterable.	s.update([4,5])

Examples

Example 1: Removing duplicates

```
numbers = [1, 2, 2, 3, 3, 4]
unique = set(numbers)
print(unique)  # {1, 2, 3, 4}
```

Example 2: Set operations

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a.union(b))
print(a.intersection(b))
print(a.difference(b))
```

6. Dictionary

Definition

A dictionary is a mutable collection of key-value pairs. Each key is unique and is used to access its corresponding value.

Properties / Characteristics

- Stores data as key-value pairs.
- Keys must be unique and hashable.
- Values may be duplicated and can be of any suitable type.
- Mutable: pairs can be added, updated, or removed.
- Preserves insertion order in Python 3.7 and later.
- Fast lookup by key is a major advantage.
- Supports nested dictionaries and complex values.

Important Methods

Method	Purpose	Example
clear()	Removes all key-value pairs.	d.clear()
copy()	Returns a shallow copy.	e = d.copy()
fromkeys(keys, value)	Creates a dictionary from keys with a common value.	dict.fromkeys(['a','b'], 0)
get(key, default)	Returns a value without raising KeyError for a missing key.	d.get('name', 'Unknown')
items()	Returns a view of key-value pairs.	d.items()
keys()	Returns a view of keys.	d.keys()
pop(key[, default])	Removes and returns a value by key.	d.pop('age')
popitem()	Removes and returns the last inserted key-value pair.	d.popitem()
setdefault(key, default)	Returns the value; inserts the key with default if missing.	d.setdefault('city', 'Hyderabad')
update(other)	Adds or changes key-value pairs.	d.update({'age': 25})
values()	Returns a view of values.	d.values()

Examples

Example 1: Creating and accessing data

```
student = {"name": "Abhinav", "age": 24, "course": "MTech"}
print(student["name"])
print(student.get("city", "Not specified"))
```

Example 2: Updating and iterating

```
student = {"name": "Abhinav", "age": 24}
student["age"] = 25
```

```
student["city"] = "Hyderabad"
for key, value in student.items():
    print(key, ":", value)
```

7. Useful Built-in Functions and Operators

Methods belong to a particular object, while built-in functions can work with many Python data types. The following functions are especially useful with the data types covered here.

Function / Operator	Purpose
len(x)	Returns the number of elements/characters.
max(x)	Returns the largest item when the values are comparable.
min(x)	Returns the smallest item when the values are comparable.
sum(x)	Adds numeric items in an iterable.
sorted(x)	Returns a new sorted list.
any(x)	Returns True if at least one item is truthy.
all(x)	Returns True if every item is truthy.
type(x)	Returns the type of an object.
in / not in	Checks membership.